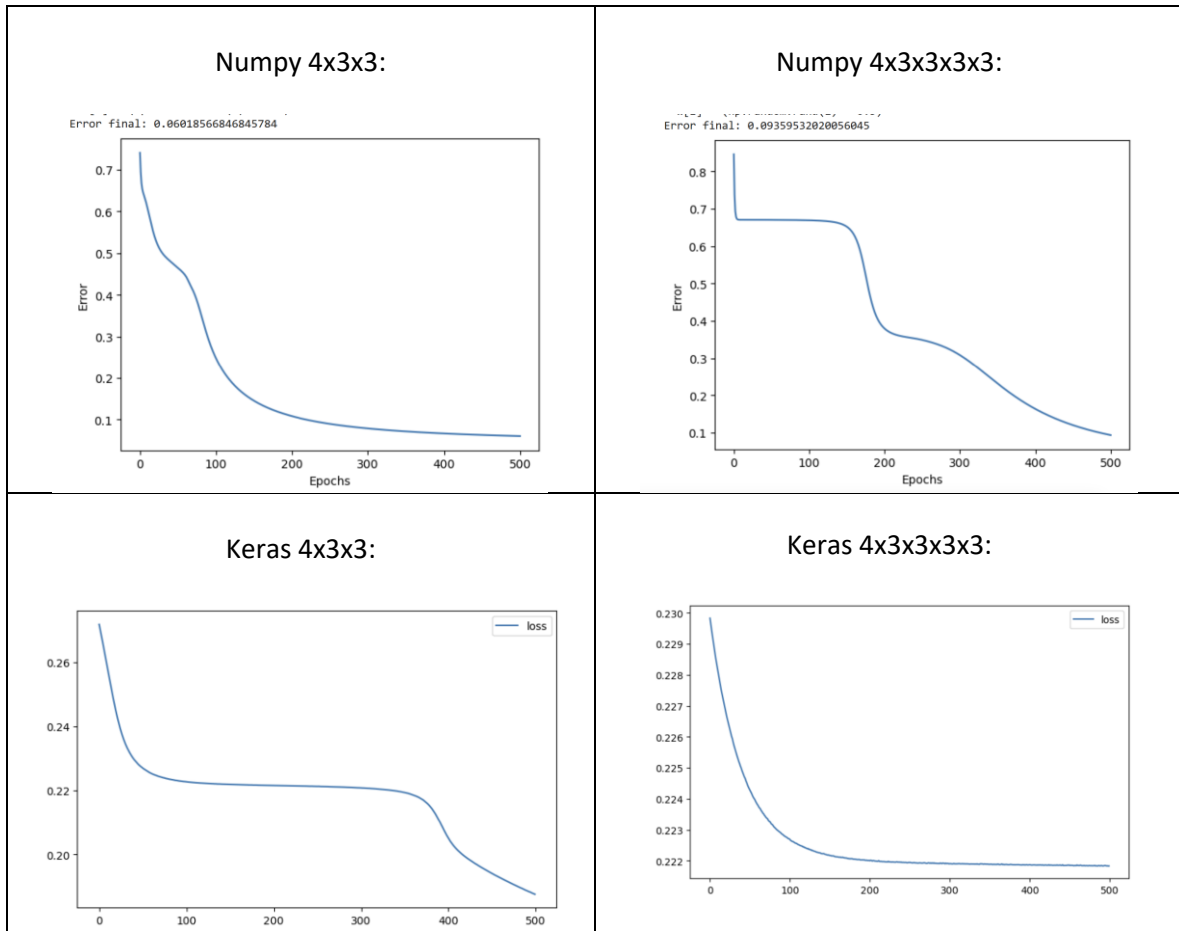


Perceptrón Multicapa – Ejercicio 01

Curvas de error/pérdida:



Reporte:

A la arquitectura original 4x3x3 se agregaron dos capas ocultas de tres neuronas, obteniendo la arquitectura 4x3x3x3x3.

Se mantuvieron los mismos parámetros: activación sigmoide en todas las capas, error MSE, descenso de gradiente estocástico (SGD), tasa de aprendizaje $\eta=0.03$ y 500 épocas.

Los resultados obtenidos son los siguientes:

Implementación	Arquitectura	Épocas	η	Error
NumPy	4x3x3	500	0.03	0.0601856
NumPy	4x3x3x3x3	500	0.03	0.0935953
Keras	4x3x3	500	0.03	0.1876604
Keras	4x3x3x3x3	500	0.03	0.2218348

Los resultados muestran que agregar dos capas no produjo una reducción del error. En NumPy el error aumentó de 0.0601856 a 0.0935953, aproximadamente un 55.5% respecto al modelo original. En Keras ocurrió algo similar, el loss aumentó de 0.1876604 a 0.2218348, aproximadamente un 18.2%. La arquitectura original tuvo un mejor comportamiento que la red más profunda tanto en NumPy como en Keras.

En NumPy la red original muestra una disminución progresiva del error, alcanzando aproximadamente 0.0601856 al final de las 500 épocas. Mientras que la red profunda presenta un periodo inicial de poco aprendizaje y posteriormente disminuye hasta aproximadamente 0.0935953.

En Keras la red original presenta una disminución rápida del loss durante las primeras épocas, luego se estanca y después vuelve a disminuir. La red profunda también disminuye inicialmente pero se estabiliza alrededor de 0.2218348 y termina con un loss superior al de la arquitectura original.

Las diferencias entre las curvas generadas por NumPy y por Keras podrían deberse a diferencias en el orden de procesamiento de los datos, la forma de realizar las operaciones de propagación y actualización de pesos y el grado de vectorización de los cálculos. También a que la implementación manual genera explícitamente los pesos utilizando NumPy, mientras que Keras utiliza su propio mecanismo de inicialización.

Aunque la arquitectura 4x3x3x3x3 tiene mayor capacidad para representar relaciones complejas, Iris es un conjunto pequeño, por lo que la arquitectura original ya tiene suficiente capacidad para aprender el problema.

Al utilizar sigmoide en todas las capas y MSE como función de pérdida, el proceso de backpropagation puede verse afectado por el desvanecimiento del gradiente. La derivada de la función sigmoide puede tomar valores pequeños y al propagarse el error a través de varias capas estos valores pueden multiplicarse y hacer que los gradientes de las primeras capas sean muy pequeños. Esto dificulta que dichas capas actualicen sus pesos de manera efectiva. Esto es lo que se observa en las curvas observadas, la red profunda de NumPy presenta un periodo prolongado de estancamiento durante las primeras épocas, mientras que la red profunda de Keras también termina estabilizándose en un loss mayor que la arquitectura original.

Notebooks:

Originales:

https://colab.research.google.com/drive/1IQD3tOXxj_26wr-OmdYvTBbM4UUIhVMr?usp=sharing

https://colab.research.google.com/drive/1CaA_pcATJcrLG8mDi68XGhCEDv1y6-ks?usp=sharing

Ejercicio:

https://colab.research.google.com/drive/1a19lkQlyPE9m-b7xbpA_YuAkR9Npn0g2?usp=sharing

<https://colab.research.google.com/drive/1gnvC-vvIPicoVdBU0fmyKR4LdGIPZyQC?usp=sharing>

Evidencias de ejecución:

